RETRO DEV KIT APPLE 1 — LE PÉCHÉ ORIGINEL

RDK BASIC — note de conception

Un clone en salle blanche de l'Integer BASIC de l'Apple-1, sans ses bugs, avec les extensions GEN2 — dédié à Uncle Bernie



VERSION 0.1 (NOTE DE CONCEPTION) 25 OCTOBRE 2026 ÉDITION FRANÇAISE

Sommaire

1 RDK BASIC : objectifs et règle de salle blanche 3
- 1.1 Pourquoi un clone ... 3
- 1.2 La règle de salle blanche ... 3
- 1.3 Compatibilité visée ... 4
- 1.4 Bannière et dédicace ... 4

2 Carte mémoire et budget 6
- 2.1 Le noyau en 4 Ko : `$E000-$EFFF` ... 6
- 2.2 Le module GEN2 ... 7

3 Instructions et fonctions 9
- 3.1 Ce que le noyau implémente ... 9
- 3.2 Jetons libres ... 10

4 Corrections des 15 bugs 11
- 4.1 Changements de comportement induits ... 12

5 Extensions GEN2 14
- 5.1 Les instructions ... 14
- 5.2 Changements côté Rust et éditeur ... 15

6 Questions pour Uncle Bernie 16

7 Plan et tests 17
- 7.1 Étapes ... 17
- 7.2 Stratégie de test ... 17

8 Licence et nom 19
- 8.1 Points incertains ... 19

CHAPITRE 1

RDK BASIC : objectifs et règle de salle blanche

RDK BASIC est un clone, écrit de zéro en assembleur 6502, de l'Integer BASIC de l'Apple-1. Il reprend sa syntaxe, ses jetons et sa disposition en mémoire, il corrige les 15 défauts documentés de l'original, et il ajoute les instructions graphiques de la carte couleur GEN2 d'Uncle Bernie. Il est dédié à Uncle Bernie.

1.1 Pourquoi un clone

L'Integer BASIC de Steve Wozniak reste sous copyright d'Apple : nous ne pouvons ni le livrer avec Retro Dev Kit Apple 1, ni le mettre dans un dépôt. Aujourd'hui, l'éditeur BASIC et les jeux portés exigent que l'utilisateur fournisse son propre `BASIC.wav`. Un clone libre lève cette limite : le kit devient complet, et on peut le distribuer et le flasher sur une vraie machine.

1.2 La règle de salle blanche

> **! RÈGLE DE SALLE BLANCHE (OBLIGATOIRE)**
>
> Personne qui écrit RDK BASIC ne **lit, ne désassemble ni ne copie** le code de l'Integer BASIC d'origine (`$E000-$EFFF`). Cela interdit de désassembler cette zone dans le débogueur du kit, de regarder un désassemblage ou un source annoté trouvé sur le web (Eric Smith, Apple II…), et de chercher des motifs dans l'image.
>
> **Sources permises :**
>
> - la spécification du comportement : les manuels Apple-1 et Apple II de l'Integer BASIC (syntaxe documentée, messages d'erreur) ;
> - nos observations en **boîte noire** : on tape des lignes, on lit l'écran et le **programme rangé en mémoire**, jamais le code de l'interpréteur ;
> - notre tokeniseur Rust `apple1_core::intbasic` , qui décrit les valeurs des jetons et le format du programme. Ce format est une **interface** : la compatibilité des cassettes en dépend, et ces valeurs ont été mesurées sur des listings tapés puis relus en mémoire ;
> - le chapitre « bugs connus » du manuel, uniquement les symptômes : on ignore la citation de désassemblage du bug 3 ;
> - l'émulation GEN2 de `apple1-core` (`gen2.rs`), le chapitre GEN2 du manuel et les tests d'Uncle Bernie.
>
> Les tests différentiels (§ 7) exécutent l'original comme une **boîte noire**. Ils comparent des sorties, jamais du code.

Chaque fichier source portera l'en-tête « clean-room implementation, written from the behaviour specification only ». Le journal Git sert de preuve de provenance.

1.3 Compatibilité visée

Point	Choix
Syntaxe	celle de l'Integer BASIC Apple-1, plus les extensions GEN2 (§ 5)
Jetons	**valeurs identiques**, contexte par contexte (§ 3). Une ligne tapée dans RDK BASIC donne en mémoire les mêmes octets que l'original : c'est vérifié par test contre `intbasic::tokenize`
Format d'une ligne	longueur (ligne entière), numéro (lo, hi), jetons, `$01` ; `:` = `$03` ; constante = `$B0 + premier chiffre`, puis valeur (lo, hi) ; noms, chaînes et `REM` en ASCII bit 7 à 1
Mémoire	programme rangé sous HIMEM en descendant, variables au-dessus de LOMEM ; pointeurs LOMEM `$4A`, HIMEM `$4C`, PP `$CA`, PV `$CC`
Cassettes	les deux plages `004A.00FF` et `PP.HIMEM-1` se chargent telles quelles, puis `E2B3R` et `RUN`
Points d'entrée	`E000R` démarrage à froid, `E2B3R` démarrage à chaud (ci-dessous)
Messages	textes identiques (`*** SYNTAX ERR`, `STOPPED AT n` …), sauf les ajouts justifiés au § 4

Points d'entrée. On garde `$E000` et `$E2B3`. Tous les guides, nos fiches de bibliothèque (`run_cmd: "E2B3R\nRUN"`), l'outil `apple1-emu basic` et les utilisateurs de vraies machines tapent ces deux adresses. Le coût est faible : un `JMP COLD` en `$E000`, et un `JMP WARM` de 3 octets placé à `$E2B3` par un segment ca65 à adresse fixe. Le code avoisinant est simplement placé autour.

- **Froid** : vide le programme (`PP = HIMEM`) et les variables, fixe LOMEM à `$0800` et HIMEM à la fin de la RAM contiguë depuis `$0000` (`$1000` en 8 Ko). Il installe le module GEN2 s'il est présent (§ 2), affiche la bannière, puis l'invite `>`.
- **Chaud** : garde les **quatre pointeurs** de la page zéro tels que la cassette les a écrits. Il réinitialise tout le reste (piles, drapeaux, vecteurs, détection GEN2) et n'affiche pas de bannière. Le bloc `$4A-$FF` d'une cassette d'origine contient en plus les octets de travail de l'original : RDK BASIC ne doit **jamais** les lire. Seuls `$4A-$4D` et `$CA-$CD` sont des données.

1.4 Bannière et dédicace

La bannière s'affiche **seulement** au démarrage à froid, juste après la ligne `E000: 4C` du moniteur. Elle utilise le jeu de caractères de l'Apple-1 : majuscules, 40 colonnes au plus, pas de minuscules ni d'accents. Les messages sont rangés comme les messages d'erreur (dernier caractère bit 7 à 1) et affichés par la même routine. L'appel coûte environ 5 octets.

Variante	Écran	Octets
A (recommandée)	`RDK BASIC 1.0` `DEDICATED TO UNCLE BERNIE`	13 + 25 + 2 CR = **40**
B	`RDK BASIC 1.0 - FOR UNCLE BERNIE`	32 + 1 = **33**
C	`RDK BASIC 1.0` `FOR UNCLE BERNIE`	13 + 16 + 2 = **31**

La variante A est la plus claire. Si le budget ROM coince en fin d'étape 5, on passe à C.

Dédicace pour le manuel (FR).

> *RDK BASIC est dédié à Uncle Bernie. Par ses travaux sur l'interface cassette ACI Gen2, par sa carte graphique couleur GEN2 et par la générosité avec laquelle il partage ses connaissances, il a rendu l'Apple-1 plus vivant que jamais, cinquante ans après sa naissance. Ce BASIC, qui sait enfin dessiner en couleur, lui doit son existence.*

Dedication for the manual (EN).

> *RDK BASIC is dedicated to Uncle Bernie. Through his work on the ACI Gen2 cassette interface, his GEN2 color graphics card, and the generosity with which he shares his knowledge, he has made the Apple-1 more alive than ever, fifty years after its birth. This BASIC, which can finally draw in color, owes its existence to him.*

CHAPITRE 2

Carte mémoire et budget

2.1 Le noyau en 4 Ko : $E000-$EFFF

Le noyau tient dans les 4 Ko de RAM en `$E000` d'un Apple-1 standard, comme l'original, qui y tient avec ses bugs. C'est le point dur du projet. Le tableau donne le budget prévu par module ; les tailles sont des estimations d'un programmeur 6502, à confirmer à l'étape 1.

Module	Contenu	Octets
Démarrage, E/S	froid/chaud, `ECHO` (`$FFEF`), lecture clavier (`$D010` / `$D011`), test d'ESC, sauts fixes	150
Messages	18 messages d'erreur, `STOPPED AT` , bannière, routine d'affichage	260
Éditeur de ligne	saisie (tampon `$0200-$027F`), `_` (effacement), insertion et suppression de lignes, `AUTO` / `MAN` , `DEL`	330
Tokeniseur	analyse dirigée par la syntaxe, table des mots-clés et des contextes	900
`LIST`	détokenisation par les mêmes tables, indentation	230
Expressions	priorités, pile d'opérandes, + − × ÷ `MOD` `^` , comparaisons, `AND` `OR` `NOT` , conversions texte ↔ entier	650
Variables et tableaux	recherche, création, `DIM` , contrôle des indices	300
Instructions	`PRINT` , `INPUT` , `LET` , `IF` , `GOTO` / `GOSUB` / `RETURN` , `FOR` / `NEXT` , `POKE` , `CALL` , `TAB` , `END` , `REM` , `RUN` , `CLR` , `NEW` , `SCR` , `CON` , `HIMEM=` , `LOMEM=`	720
Chaînes	`DIM` , affectation, sous-chaînes, comparaison, `LEN` , `ASC`	330
Extensions	crochets et vecteurs GEN2, vecteur `USR` , bloc de signature	90
Total		**≈ 3 960 / 4 096**

La marge est d'environ 130 octets. Si elle disparaît, trois leviers, dans cet ordre :

1. factoriser l'analyse des instructions à motif commun (`expr , expr AT expr`) ;
2. passer la bannière en variante C ;
3. retirer `AUTO` / `MAN` du noyau et les mettre dans le module GEN2.

RAM utilisée par le noyau (en dehors du programme et des variables) :

Zone	Usage
`$4A-$FF`	pointeurs de l'interface (`$4A` , `$4C` , `$CA` , `$CC`), et octets de travail réinitialisés à chaque démarrage
`$0A-$0C`	vecteur `USR` : `JMP adresse` (convention Applesoft), installé au froid sur une routine qui rend 0
`$0100-$017F`	piles BASIC : 8 `FOR` de 10 octets, 8 `GOSUB` de 4 octets, et une pile d'opérandes. La pile 6502 garde `$0180-$01FF`
`$0200-$027F`	tampon de la ligne tapée, comme le Woz Monitor
sous PP	ligne directe tokenisée (au plus 128 octets), prise dans l'espace libre : pas de zone fixe

Le noyau n'utilise ni `$24-$2B` (Woz Monitor), ni `$0280-$07FF` : c'est la zone « libre pour le langage machine » du manuel, qui reste libre.

2.2 Le module GEN2

La carte GEN2 apporte la RAM `$0000-$BFFF` . Ses pages vidéo sont dans cette RAM :

Mode	Page 1	Page 2
texte et LORES	`$0400-$07FF`	`$0800-$0BFF`
HIRES	`$2000-$3FFF`	`$4000-$5FFF`

Le module (environ 1,3 Ko) est placé **en haut de la RAM de la carte, en `$B000-$B7FF`** ; `$B800-$BFFF` reste réservé à une seconde extension. Il commence par un en-tête :

```
$B000  "RDKX"           signature
$B004  version, taille
$B006  adresse de la table des actions (une par jeton GEN2)
$B008  adresse de la routine d'installation
```

Au démarrage à froid comme à chaud, le noyau cherche `RDKX` en `$B000` . Si l'en-tête y est et que la carte est présente (`$B000` est de la RAM), il retient la table des actions. La recherche est refaite à chaque démarrage, si bien qu'aucun état n'est perdu quand une cassette écrase la page zéro.

Crochets. Pour que l'édition et `LIST` marchent même sans la carte, la **syntaxe et les noms** des mots-clés GEN2 sont dans le noyau : environ 90 octets de table. Seule **l'exécution** passe par le module : un jeton GEN2 rencontré pendant `RUN` saute à l'entrée de la table des actions. Sans module, l'erreur est `*** GEN2 ERR` , un nouveau message de 4 octets. Un programme graphique reste donc éditable et sauvegardable sur un Apple-1 de 8 Ko.

HIMEM, LOMEM et pages vidéo. Quand le module est installé, le démarrage à froid choisit :

- **HIMEM = `$B000`** , juste sous le module ;
- **LOMEM = `$6000`** , au-dessus des deux pages HIRES. Le programme et les variables disposent alors de 20 Ko (`$6000-$AFFF`), et toutes les pages vidéo `$0400-$5FFF` restent libres.

Un utilisateur sans HIRES peut taper `LOMEM=3072` (`$0C00`, après les pages LORES) et gagner 13 Ko. Chaque commande graphique vérifie que la page qu'elle touche ne recoupe pas `[LOMEM, HIMEM)`, sinon `*** MEM FULL ERR`. Les cassettes d'origine portent leur LOMEM/HIMEM (`$0800` / `$1000`) : elles tournent comme avant, mais leurs variables sont dans la page 2 LORES. On ne peut pas y utiliser `PAGE 2` en LORES, et le contrôle le signale.

CHAPITRE 3

Instructions et fonctions

3.1 Ce que le noyau implémente

Commandes du mode direct (jamais rangées dans un programme) :

- `LIST [a[,b]]`, `RUN [n]`, `CLR`, `NEW`, `SCR`, `CON` (reprise après `STOPPED AT`) ;
- `AUTO a[,pas]` et `MAN` (numérotation automatique) ;
- `DEL a,b` ;
- `HIMEM=expr`, `LOMEM=expr`.

Instructions, avec leur jeton d'instruction dans `intbasic` :

Instruction	Jeton	Instruction	Jeton
`REM`	`$5D`	`FOR … = … TO … STEP`	`$55` `$56` `$57` `$58`
`LET` (et affectation implicite)	`$5E` ; `=` `$71`, `$70` pour une chaîne	`NEXT v[,v]`	`$59` `$5A`
`DIM`	`$4F` (nombre), `$4E` (chaîne) ; suite `$44` / `$43`	`RETURN`	`$5B`
`TAB`	`$50`	`GOSUB`	`$5C`
`END`	`$51`	`GOTO`	`$5F`
`INPUT`	`$54` / `$52` / `$53` (avec invite) ; suite `$27` / `$26`	`IF … THEN`	`$60` ; `THEN` n° `$24`, `THEN` instr. `$25`
`PRINT`	`$62` / `$61` / `$63` seul ; `;` `$45` / `$46`, `,` `$48` / `$49`, fin `$47`	`POKE a,v`	`$64` `$65`
`CALL`	`$4D`	`COLOR=` `PLOT` `HLIN`	`$66` ; `$67` `$68` ; `$69` `$6A` `$6B`

Fonctions :

- `PEEK` `$2E`, `RND` `$2F`, `SGN` `$30`, `ABS` `$31`, `USR` `$32`, avec `(` `$3F` ;
- `LEN(` `$3B` ;
- nouveaux jetons : `ASC(` `$3C`, `SCRN(` `$3D`, et `,` de `SCRN` `$3E`.

Opérateurs :

- binaires : `+` `$12`, `-` `$13`, `*` `$14`, `/` `$15`, `=` `$16`, `#` `$17`, `>=` `$18`, `>` `$19`, `<=` `$1A`, `<>` `$1B`, `<` `$1C`, `AND` `$1D`, `OR` `$1E`, `MOD` `$1F`, `^` `$20` ;
- unaires : `+` `$35`, `-` `$36`, `NOT` `$37` ;
- parenthèse `$38`, `)` `$72` ;

- chaînes : `=` `$39`, `#` `$3A`, guillemets `$28` / `$29`, sous-chaîne `$2A` / `$23`, indice de tableau `$2D`, `$` `$40`, indice de chaîne en affectation `$42`.

Les priorités, la division tronquée vers zéro, le signe de `MOD`, les booléens 0/1, les noms (lettre + chiffre facultatif, chaînes d'une lettre), les tableaux indexés de 1 à n et les chaînes de 255 caractères au plus suivent les chapitres BASIC du manuel complet (référence, mémoire, erreurs).

3.2 Jetons libres

Les valeurs ci-dessous ne sont produites ni par `intbasic`, ni par aucune ligne acceptée par l'original. Nous l'avons vérifié en boîte noire le 25 octobre 2026 : les lignes `COLOR=3`, `PLOT 1,2` et `HLIN 1,2 AT 3`, rangées puis relues en mémoire, donnent `$66`, `$67 … $68 …` et `$69 … $6A … $6B …`. `VLIN`, `VTAB`, `ASC(`, `SCRN(` sont refusés, et rien n'est rangé. Aucun programme d'origine ne contient donc ces octets :

`$00`, `$02`, `$04-$11`, `$21`, `$2B`, `$2C`, `$33`, `$3C-$3E`, `$41`, `$4A-$4C`, `$6C-$6F`, `$73-$7F`.

RDK BASIC prend `$3C-$3E` pour les fonctions, et `$6C-$6E` et `$73-$7B` pour les instructions GEN2 (§ 5). `$6F` reste réservé (`VTAB`), et `$7C-$7F` restent libres.

CHAPITRE 4

Corrections des 15 bugs

Il n’y a pas de mode « bugs fidèles ». La décision vient de l’utilisateur : très peu de programmes Integer BASIC existent, et la différentielle (§ 7) trouvera ceux qui dépendaient d’un bug.

#	Symptôme d'origine	Dans RDK BASIC
1	sortir d'un `FOR` par `GOTO` laisse l'entrée sur la pile, et `>8 FORS ERR` finit par arriver	`FOR v` cherche d'abord une entrée de même variable dans la pile, et la supprime avec tout ce qui est au-dessus (règle du BASIC Microsoft)
2	`^` bloque la machine	puissance entière par élévation au carré, avec contrôle de dépassement (`>32767 ERR`). `0^0 = 1` . Un exposant négatif donne 0, sauf pour une base ±1 (troncature)
3	pile d'opérandes pleine : message `PPED AT` (mauvais index de message)	le bon message, `*** TOO MANY PARENS ERR`
4	`IF … THEN a:b` exécute `b` même si la condition est fausse	**décidé** : une condition fausse saute **tout le reste de la ligne**, `:` compris. Pas de mode « bug fidèle »
5	−32768 est inutilisable	plage complète −32768…32767. `-(-32768)` , `ABS(-32768)` et `-32768/-1` donnent `>32767 ERR`
6	`ASC(` et `SCRN(` sont refusés	`ASC(a$)` donne le code du 1er caractère, bit 7 à 0 (`ASC("")` : `STRING ERR`) ; `SCRN(x,y)` via le module GEN2
7	`COLOR=` , `PLOT` , `HLIN` sans effet ; `VLIN` , `VTAB` refusés	ils dessinent avec le module GEN2 (§ 5), sinon `GEN2 ERR` ; `VTAB` reste refusé (le terminal de l'Apple-1 ne positionne pas le curseur)
8	(Apple II) plantage sur les lignes de 251 caractères	ligne tapée limitée à 127 caractères ; au-delà, les touches sont ignorées et RETURN donne `TOO LONG ERR` . Une ligne tokenisée de plus de 255 octets est refusée
9	`STOPPED AT 65535` après une erreur de saisie ou en mode direct	un drapeau « en exécution » : en mode direct, pas de `STOPPED AT`
10	`USR(x)` saute en `$0000` et plante	`USR` appelle le vecteur `$0A` (`JMP`) avec x dans A (lo) et Y (hi), et rend A/Y. Au froid, le vecteur pointe sur une routine qui rend 0
11	`HIMEM=` plus bas est ignoré sans message	accepté si le programme et les variables tiennent. Le bloc programme, qui ne contient aucune adresse absolue, est déplacé (copie mémoire), puis PP et HIMEM mis à jour, sinon `MEM FULL ERR`
12	`:` refusé en mode direct, `FOR` et `GOSUB` aussi	ligne directe tokenisée sous PP, puis exécutée comme une ligne de programme ; `FOR … : NEXT` marche sur une ligne directe. Un `GOTO` depuis le mode direct lance le programme à ce numéro
13	`INPUT "AGE ? ",N` affiche `AGE ? ?`	`?` ajouté seulement si l'invite ne finit pas déjà par `?` (espaces de fin ignorés). Sans invite : `?` , comme avant
14	`FOR I=5 TO 1` exécute le corps une fois	test en entrée de boucle selon le signe du pas. S'il échoue, on saute après le `NEXT v` correspondant (recherche avec comptage des `FOR` / `NEXT` imbriqués). Pas de `NEXT` : `BAD NEXT ERR`
15	n'importe quelle touche arrête le programme, donc pas de jeu en temps réel	seule **ESC** arrête (`STOPPED AT n`). Les autres touches restent dans le PIA : `PEEK(-12271)` (`$D011` , bit 7 = touche prête) et `PEEK(-12272)` (`$D010` , la lit et l'acquitte)

4.1 Changements de comportement induits

- **Bug 4 (décidé).** Une condition fausse saute désormais le reste de la ligne, comme dans presque tous les BASIC ; il n'y a pas de mode « bug fidèle ». C'est le seul changement qui peut casser un programme existant. Le motif `IF cond THEN n : instr` comptait sur le bug pour exécuter

`instr` quand la condition est fausse. L'exemple « rang d'une lettre » du manuel en dépend : `IF C$(K,K)=L$ THEN 60:NEXT K`. Trois mesures :

- un **avertissement** dans l'éditeur du kit (et dans `apple1-emu basic --rdk`) dès qu'un programme chargé ou tapé contient des instructions après un `:` qui suit un `IF … THEN` : leur comportement diffère de l'Integer BASIC d'origine ;
- un audit de la bibliothèque par la différentielle ;
- la mise à jour de l'exemple du manuel en même temps que la sortie de RDK BASIC.

- **Bug 15.** Une touche autre qu'ESC n'arrête plus un programme. Le manuel doit le dire.
- **Bug 13.** Une invite qui finit par `?` ne reçoit plus de second `?`.
- **Bug 14.** Une boucle vide n'est plus parcourue.
- **Bugs 2, 5, 6, 10, 11, 12.** Ce qui donnait une erreur ou un plantage fonctionne désormais. Aucun programme correct ne change.
- **Nouveaux messages.** `GEN2 ERR` est nouveau. `TOO MANY PARENS ERR` existait dans la table d'origine sans être visible.
- **Gardé tel quel**, car ce n'est pas un bug : `*** END ERR` quand un programme se termine sans `END`, les comparaisons de chaînes limitées à `=` / `#`, `A(0)` en `RANGE ERR`, `RND(0)` en `>32767 ERR`.

CHAPITRE 5

Extensions GEN2

5.1 Les instructions

Les commutateurs de la carte sont en `$C250-$C257` et **réagissent en lecture** : le module utilise `BIT $C25x`, jamais `STA`. Le texte que le programme affiche par `PRINT` reste sur le terminal de l'Apple-1. Le moniteur couleur ne montre que les pages de la carte.

Instruction	Jeton	Action sur la GEN2
`GR`	`$73`	lit `$C250` (graphique), `$C256` (LORES), `$C252` (plein écran) et la page d'affichage ; efface la page LORES (couleur 0) ; `COLOR=0`
`TEXT`	`$74`	lit `$C251` (texte)
`COLOR=n`	`$66`	couleur LORES 0-15 (ordre de la table GEN2 du manuel : 0 noir… 15 blanc)
`PLOT x,y`	`$67` `$68`	x 0-39, y 0-47 : octet de la rangée `y/2`, en `base + (r&7)×$80 + (r>>3)×$28 + x`. Quartet bas si y est pair, haut sinon
`HLIN x1,x2 AT y`	`$69` `$6A` `$6B`	ligne horizontale LORES
`VLIN y1,y2 AT x`	`$6C` `$6D` `$6E`	ligne verticale LORES
`SCRN(x,y)`	`$3D` `$3E` `( )` `$72 )`	couleur du bloc (fonction)
`HGR`	`$75`	`$C250`, `$C257` (HIRES), `$C252`, page ; efface la page HIRES (8 Ko) ; vérifie qu'elle est hors de `[LOMEM, HIMEM)`
`HCOLOR=n`	`$76`	couleur HIRES 0-7, comme Applesoft : 0 noir, 1 vert, 2 violet, 3 blanc, 4 noir, 5 orange, 6 bleu, 7 blanc. Bit 7 = palette (4-7), et motif `$2A` / `$55` selon la parité de la colonne
`HPLOT x,y [TO x,y …]` et `HPLOT TO x,y`	`$77` , `$78` , `TO` `$79`	point ou segments (Bresenham). x 0-279, y 0-191, ligne `$2000 + (y&7)×$400 + ((y>>3)&7)×$80 + (y>>6)×$28` (+ `$2000` en page 2), 7 points par octet, bit 0 à gauche
`PAGE d[,w]`	`$7A` , `$7B`	page affichée d (`$C254` / `$C255`) et page de dessin w (par défaut = d), pour le double tampon

Hors plage, on a `RANGE ERR` (coordonnées) ou `>255 ERR` (couleur). Le mode mixte (`$C253`) affiche les 4 dernières lignes de la **page texte GEN2**, que `PRINT` n'alimente pas : on ne l'active pas par défaut. Il reste accessible par `X=PEEK(-15789)` . Le module garde sa couleur LORES, sa couleur HIRES, la dernière position `HPLOT` et la page de dessin dans sa propre RAM (`$B7F0-$B7FF`), pas en page zéro.

Taille estimée du module. Environ 1,3 Ko :

Partie	Octets
en-tête et table des actions	60
calcul d'adresse des rangées LORES (sans table)	40
`PLOT` , `HLIN` , `VLIN` , `SCRN`	200
`GR` , `TEXT` , `HGR` , effacement	120
calcul d'adresse HIRES et masques de points	180
`HPLOT` avec segments	350
`HCOLOR` , `PAGE` , contrôles	150
marge	200

5.2 Changements côté Rust et éditeur

- `apple1_core::intbasic` : ajouter un **dialecte** (`Dialect::Apple1` ou `Dialect::Rdk`).
  - En mode RDK, le tokeniseur accepte `ASC(` , `SCRN(` , `VLIN` , `GR` , `TEXT` , `HGR` , `HCOLOR=` , `HPLOT` , `PAGE` , avec les jetons ci-dessus.
  - Il signale (avertissement, pas erreur) les lignes `IF … THEN … : …` , dont le comportement diffère de l'original (bug 4).
  - Il accepte −32768 comme constante.
  - Le dialecte Apple-1 reste la référence pour les cassettes destinées au vrai BASIC.
- `KEYWORDS` : ajouter les nouveaux mots pour la coloration de l'éditeur du kit et du lexer Pygments de la documentation.
- Détection du BASIC : `debug::basic_present` , la détection de la session GUI et l'outil MCP `load_basic` reconnaissent l'original par des octets fixes. Pour RDK BASIC, ajouter un **bloc de signature** documenté en `$EFF0` : `"RDKB"` , version, adresse du crochet « nouvelle ligne », adresse où se trouve le pointeur de ligne courante. Le débogueur BASIC (pas à pas par ligne, points d'arrêt sur ligne, variables) les lit de là.
- Bibliothèque : RDK BASIC devient le BASIC par défaut, livré et libre. `BASIC.wav` d'origine reste possible, par une préférence.

CHAPITRE 6

Questions pour Uncle Bernie

(*En anglais, pour un message privé.*)

1. **RAM for the extension**: we plan to load the GEN2 BASIC extension at `$B000-$B7FF`, with HIMEM = `$B000` and LOMEM = `$6000` so both HIRES pages stay free. Is there a RAM area you would recommend instead, or one we should avoid on the real card?
2. **Colors**: for `COLOR=` we use the Apple II LORES order 0–15, and for `HCOLOR=` the Applesoft order 0–7 (black, green, violet, white, black, orange, blue, white). Does the GEN2 output match these, especially colors 5 and 10 (both grey)?
3. **Firmware space**: do you plan any ROM/EPROM socket or firmware area on the card? An on-card copy of the extension would avoid loading it from tape.
4. **Mixed mode**: the bottom four text lines show the GEN2 text page, which Apple-1 `PRINT` does not write. Would you like a `GPRINT`-style statement for the GEN2 text page, or is that out of scope?
5. **Bundling**: would you like RDK BASIC (MIT licence) to be bundled with, or recommended for, the GEN2 card?
6. **Dedication**: RDK BASIC is dedicated to you for your work on the ACI Gen2 and the GEN2 color card. Are you happy with that, and how would you like to be named in the banner and the manual (“Uncle Bernie”, your real name, or both)?

CHAPITRE 7

Plan et tests

7.1 Étapes

Étape	Contenu	Taille estimée
1	démarrage froid/chaud, E/S, messages, éditeur de ligne, tokeniseur (instructions de l'étape), `LIST`, expressions, `PRINT`, `LET`, `GOTO`, `IF`, `END`, `REM`, `RUN` / `NEW` / `CLR` / `SCR`, variables simples	2,3 Ko
2	`FOR` / `NEXT`, `GOSUB` / `RETURN`, `INPUT`, `DIM` et tableaux, `POKE`, `PEEK`, `CALL`, `TAB`, `RND`, `SGN`, `ABS`, `USR`, `AUTO` / `MAN` / `DEL`, `CON`, `HIMEM=` / `LOMEM=`	+1,0 Ko (3,3)
3	chaînes : `DIM`, affectation, sous-chaînes, comparaison, `LEN`, `ASC`, `INPUT` chaîne	+0,35 Ko (3,65)
4	module GEN2 en `$B000`, crochets du noyau, signature `RDKX`	1,3 Ko (module) + 0,1 Ko (noyau)
5	finition : corrections restantes, bannière, signature `RDKB` et débogueur, dialecte Rust, alerte `IF`, audit de la bibliothèque, chapitre du manuel	gain de place visé : 0,1-0,2 Ko

Chaque étape produit une image utilisable, chargée en `$E000` par la bibliothèque du kit ou par `apple1-mcp reset {basic_path}`. Elle est assemblée par `apple1_core::asm6502`, avec un listing et des symboles pour le débogueur.

7.2 Stratégie de test

1. **Format.** Pour chaque ligne du corpus, on tape la ligne dans RDK BASIC, on relit la mémoire, et on compare **octet par octet** à `intbasic::tokenize`. Même chose pour `LIST`, qui doit redonner le texte normalisé.
2. **Différentielle.** Le même programme et les mêmes entrées clavier tournent sur deux sessions, pilotées par le serveur `apple1-mcp` ou directement par `apple1_core::Machine` dans des tests Rust :
   - **l'original**, chargé depuis `/tmp/gt2/lib/BASIC.wav`. **Test seulement**, jamais commité ni livré : le test est sauté si le fichier est absent ;
   - **RDK BASIC**.

   On compare les écrans et les messages. Les écarts attendus (les 15 corrections) sont listés dans un fichier d'exceptions par programme : tout autre écart fait échouer le test. 3. **Unitaires.** Un test par instruction et par erreur (`*** … ERR`, `STOPPED AT`), y compris aux limites : −32768, 32767, 8 `FOR`, 8 `GOSUB`, 12 parenthèses, chaînes de 255, `DIM` en `MEM FULL`. Un test par bug corrigé, qui reprend l'exemple du chapitre des bugs. 4. **GEN2.** Les tests dessinent, puis lisent la mémoire vidéo et les commutateurs dans `Gen2` : `PLOT` donne le bon quartet, `HPLOT 0,0 TO 279,191` donne la

bonne diagonale. Le moniteur couleur sert aux captures. 5. **Corpus.** Les exemples `docs/exemples/basic` et ceux des chapitres BASIC du manuel, les programmes BASIC de la bibliothèque du kit, les jeux portés (chapitre « portage »), et les programmes ASCII générés par `apple1-emu ascii --format basic`. 6. **Taille.** Un test échoue si le noyau dépasse `$EFEF`, sous la signature `RDKB`.

CHAPITRE 8

Licence et nom

Nom. RDK BASIC (« Retro Dev Kit BASIC ») : court, lisible dans la bannière, et sans les mots « Apple » ni « Integer BASIC », qui pourraient faire croire à une copie d'Apple. Le manuel le présente comme « un BASIC entier compatible avec l'Integer BASIC de l'Apple-1 ». Le module s'appelle **RDK BASIC GEN2**.

Licence. MIT, comme le reste du projet (`license = "MIT"` dans `Cargo.toml`) : simple, et compatible avec une diffusion sur EPROM, en kit ou par Uncle Bernie avec sa carte. Une double licence MIT OR Apache-2.0 ajouterait une clause de brevets, inutile ici. On mentionne la dédicace et la règle de salle blanche dans le fichier `LICENSE`, ou dans un `NOTICE` à côté.

8.1 Points incertains

- **Le budget de 4 Ko** reste une estimation. L'étape 1 doit mesurer le tokeniseur, le plus gros module, avant de figer le reste.
- **Les jetons libres** sont établis sur notre tokeniseur et sur l'essai en boîte noire du 25 octobre. Une instruction acceptée par l'original que nous n'aurions jamais tapée pourrait produire un jeton de la liste. On complétera l'essai avec les lignes du manuel Apple II avant de figer les valeurs.
- **Les commandes** `AUTO`, `MAN`, `DEL`, `CON` sont décrites d'après les manuels. Leurs détails d'affichage (format de `AUTO`) seront relevés en boîte noire.